

# SOI CERTIFICATE MANAGEMENT PLATFORM

## 12-PROMPT IMPLEMENTATION PLAYBOOK

Copy-ready AI development prompts, scope boundaries, tenant rules, UI requirements, test gates,  
Update Center ZIP requirements and post-install verification

### Build model

One prompt -> one cumulative ZIP -> install -> verify -> repair if needed -> only then continue. The planned v1 scope is complete after Prompt 12 passes its final acceptance gate. Repair prompts such as 4.1 or 9.1 are corrective releases, not extra roadmap scope.

*Companion to the SOI Certificate Management Platform Developer Plan*

Prepared for staged AI-assisted development and production verification

## How to Use This Prompt Pack

This document contains 12 planned build prompts. Run them in order. Each prompt assumes the prior prompt is installed, verified and represented accurately in docs/development-status.md. Do not skip a failed exit gate. If testing exposes a defect, issue a repair prompt named N.1 that fixes the defect, produces a new cumulative ZIP, reruns the full suite, and updates the handoff document before moving forward.

- For every main prompt, give the coding AI access to the existing repository/codebase and the latest development-status.md.
- Ask the AI to establish ground truth from code, migrations, package metadata and tests before editing. Prior reports are useful context but are not authoritative over the repository.
- Install the generated cumulative ZIP through the SOI CMS Update Center and perform the Owner Verification checklist from this document.
- Do not carry known critical or high-severity defects into the next prompt. Use a repair prompt instead.
- The latest ZIP must be sufficient for a fresh installation or supported upgrade; it must not depend on installing all prior ZIPs in sequence.
- The product stays fully self-hosted in v1. Cloudflare, AWS, Redis and external queues are future adapters, not runtime dependencies.

## Global Architecture Contract

- Platform type: modular, multi-tenant SaaS-style application running as a plugin/product on the existing SOI CMS used for MyApps.
- Deployment: web-only through the CMS Update Center; no SSH or CLI required for install, upgrade, migration, tests or normal operations.
- Primary stack: existing PHP runtime, MySQL, CMS authentication/session/CSRF, local filesystem storage and a locally available/bundled PDF + QR implementation.
- No mandatory Redis, message broker, external queue, Cloudflare, AWS, object storage or remote rendering service in v1.
- Domain is always softcoded from CMS/request configuration. Never hardcode a production hostname.
- Routes: /super-admin for platform control, /manage for tenant administration, /console for operational issuance, /docs for documentation, /verify/{token} for verification.
- Tenancy: shared application/codebase and database with strict row-level tenant isolation. Every tenant-owned object has tenant\_id and every repository/service query constrains it.
- Platform access: CMS platform administrators may act as certificate platform administrators, but bypass logic must be explicit and audited. Tenant roles never imply platform access.
- Tenant roles: Owner, Administrator, Template Designer, Issuer, Viewer, with granular permission keys underneath and room for custom roles later.
- Issued certificates are authoritative records. Normal UI never hard-deletes an issued certificate; lifecycle state changes preserve history.
- All issuance channels converge on one CertificateIssuanceService. Manual, form, API, scheduled/automatic and bulk flows must not implement separate issuance business rules.
- Templates are versioned. Issued certificates reference an immutable template/version snapshot so later template edits cannot alter historical output.
- Every certificate has a human-friendly number plus a separate high-entropy verification token. The QR contains only the verification URL/token reference, never sensitive certificate data.
- Rendering, storage and later external integrations sit behind interfaces so future adapters can be added without rewriting certificate domain logic.
- All security-sensitive state changes are auditable. Secrets are never logged in clear text. API secrets are one-time reveal and stored using a secure hash or equivalent safe comparison method.
- UI must be enterprise-grade, responsive and usable on desktop, tablet and mobile management surfaces; the designer canvas may use controlled scaling/scrolling but must not break the shell.
- Avoid debug/status footer strips that consume page height. Health/version information belongs in dedicated admin areas.

## Universal UI Verification Matrix

| Verification area | Required check                                               | Pass condition                                                                              |
|-------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Desktop shell     | Verify affected pages around 1440 x 900.                     | No overlap, clipped controls, accidental horizontal page scroll or oversized empty regions. |
| Ultra-wide        | Verify around 1920 x 1080 and, where practical, 2560 x 1440. | Content remains readable; max-width/layout adapts; tables and dashboards do not             |

| Verification area   | Required check                                                           | Pass condition                                                                                  |
|---------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|                     |                                                                          | become excessively sparse.                                                                      |
| Tablet              | Verify around 768 x 1024.                                                | Navigation, tables, filters, modals and forms remain usable; actions are not hidden off-screen. |
| Mobile              | Verify around 390 x 844 for management pages intended to work on mobile. | Controls stack cleanly; labels remain associated with fields; no unusable hover-only actions.   |
| Keyboard/focus      | Tab through primary actions and modal/forms.                             | Visible focus, logical order, no keyboard trap, Escape/cancel behavior where appropriate.       |
| Empty/error/loading | Force empty data, validation errors and server error states.             | Human-readable states; no raw exception, SQL, path or stack trace leakage.                      |
| Authorization UI    | Open URL directly as users with and without access.                      | UI hiding is not the security boundary; server returns correct denial and no protected data.    |
| Consistency         | Compare /super-admin, /manage and /console shells.                       | Consistent header/navigation spacing, typography, controls and tenant identity treatment.       |

## Common Release Gate Required After Every Prompt

- ☐ New/changed PHP files lint cleanly using the project's available web-safe validation path.
- ☐ New migrations are idempotent or version-gated and upgrade from the previous ZIP succeeds without manual SQL.
- ☐ Full available automated test suite passes, not only new tests.
- ☐ HTTP/browser smoke checks pass for all affected routes.
- ☐ Cross-tenant negative tests exist for every new tenant-owned resource.
- ☐ CSRF is enforced on state-changing browser actions; API actions use API authentication/scopes, not CSRF.
- ☐ No secrets, raw stack traces, SQL errors, local filesystem paths or sensitive payloads appear in user-visible error output.
- ☐ One cumulative Update Center ZIP is produced and its manifest/layout are validated.
- ☐ Active plugin installations remain active after upgrade; do not assume a fresh Update Center install automatically activates the plugin.
- ☐ docs/development-status.md is updated with exact versions, schema, routes, tests, completed/partial/pending work, known defects and next prompt guidance.
- ☐ If a critical/high-severity defect remains, the phase is reported as incomplete and no misleading production-ready claim is made.

## Roadmap at a Glance

| Prompt | Name                                              | Exit outcome                                                                            |
|--------|---------------------------------------------------|-----------------------------------------------------------------------------------------|
| 1      | Platform Foundation & Modular Bootstrap           | Installable certificate product shell with modules, migrations, health and base routes. |
| 2      | Multi-Tenancy, Identity & RBAC                    | Tenant isolation, memberships, roles, permissions and portal authorization.             |
| 3      | Template Domain, Assets, Variables & Versioning   | Reliable template data model and immutable published versions.                          |
| 4      | Visual Certificate Designer & Preview             | Enterprise visual editor producing validated structured template definitions.           |
| 5      | Local Rendering, QR & Artifact Storage            | Fully self-hosted PDF/QR rendering and safe local artifact storage.                     |
| 6      | Issuance Engine, Registry & Verification          | End-to-end issuance, authoritative registry and verification policy/modes.              |
| 7      | Lifecycle, Audit, Search & Operational Management | Revoke/expire/replace/cancel, audit trails and certificate operations.                  |

| Prompt | Name                                                                  | Exit outcome                                                                 |
|--------|-----------------------------------------------------------------------|------------------------------------------------------------------------------|
| 8      | Manual Issuance, Dynamic Forms, Automation & Scheduling               | Human, form and scheduled/rule-driven triggers through one issuance service. |
| 9      | API, Authentication, Idempotency, Webhooks & Integrations             | Tenant-scoped external API and integration contract.                         |
| 10     | Bulk Issuance & Imports                                               | Large CSV/XLSX/paste imports with chunked progress and row-level outcomes.   |
| 11     | Dashboards, Branding, Documentation & Admin Operations                | Polished portals, docs, health, search/export and tenant settings.           |
| 12     | Production Hardening, Security, Performance, Recovery & Final Release | Full regression, migration, security, browser and release acceptance.        |

**PROMPT 1**

## Platform Foundation & Modular Bootstrap

**Target release**

Suggested product version 1.0.0. Exit outcome: The product installs as a valid SOI CMS plugin, exposes safe base routes, owns a versioned schema/migration system, and has a modular core ready for certificate features.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

**Operating instructions**

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Build the certificate platform foundation only. Do not prematurely implement the full designer, issuance API or certificate lifecycle. The purpose of this prompt is to create a dependable platform skeleton that later prompts can extend without rewriting bootstrapping, routing, migrations or module discovery.

**Required implementation**

- Create/normalize plugin slug and namespace, recommended as certificates / SOI\Certificates (or follow the CMS naming convention already used by MyApps if a different canonical convention exists).
- Create a small autoload/bootstrap layer compatible with the CMS and current PHP version. Do not introduce a framework or deployment-time Composer requirement.
- Implement a module registry/interface so product modules can declare routes, permissions, migrations, navigation entries, docs and health checks without putting all logic in plugin.php.
- Implement product version and schema version tracking using the CMS option/config system. Schema version changes only when database structure changes.
- Implement a migration registry/runner that is safe through web requests, records applied migrations, prevents double application and surfaces failures to authorized admins without leaking SQL internals.
- Create a health service with at least plugin version, schema version, database connectivity, migration state and local storage base-path writability checks. The health detail is admin-only.
- Register base route shells for /super-admin, /manage, /console and /docs. At this stage they may show a clean foundation screen, but must be real controllers/views and must not be dead routes.
- Create a public route namespace for /verify/{token} but return a neutral not-found/feature-not-yet-available response until verification is implemented. Do not expose internals.
- Add shared view/layout helpers, flash/validation messaging, CSRF helper integration, route URL helpers and asset loading that respect a softcoded base URL.
- Create initial audit infrastructure interface/service and database table if appropriate, even if only platform bootstrap events are logged in this phase.
- Create docs/development-status.md plus a short /docs architecture page explaining routes, versions and current phase status.
- Provide web-accessible test/smoke entry points only if consistent with existing CMS practices; protect them so they cannot expose internals to ordinary users in production.

**Recommended initial module boundaries**

Core/  
Platform/

|                |                                             |
|----------------|---------------------------------------------|
| Tenancy/       | (placeholder registration only in Prompt 1) |
| Authorization/ | (foundation only)                           |
| Audit/         |                                             |
| Templates/     | (module stub, no business features yet)     |
| Rendering/     | (interface placeholder only)                |
| Issuance/      | (interface placeholder only)                |
| Verification/  | (route placeholder only)                    |
| Api/           | (not active yet)                            |
| Docs/          |                                             |

## Database and migration requirements

- Use the CMS table-prefix mechanism exactly once; guard against accidental double prefixing.
- Create only foundation tables needed now, such as migration/version metadata and audit events if the CMS does not already provide them.
- All migrations must be reversible where practical or at minimum safely retryable/version-gated. A failed migration must not silently mark itself applied.
- Provide an authorized web migration status screen or equivalent within /super-admin so deployment does not require SQL shell access.

## UI requirements

- Use an enterprise shell consistent with the CMS/MyApps visual language but keep certificate product branding distinct.
- Navigation placeholders may show planned modules as disabled only if clearly labeled; do not create clickable dead links.
- No giant hero cards, debug footer or version strip on every page. Keep operational metadata in a dedicated About/Health area.
- Responsive shell and accessible focus states are required from the first prompt so later pages inherit them.

## Automated verification required before packaging

- Bootstrap can load repeatedly in one request without duplicate route/module registration.
- Migration runner applies pending migration once and reports current schema version correctly.
- Database prefixing creates/queries the expected table names.
- Base routes return expected status for authorized/unauthorized users and do not fatal.
- Softcoded URL generation works under a non-root path/domain configuration if the CMS supports it.
- Upgrade package does not deactivate an already active plugin.

| Verification area | Required check                                                                                                              | Pass condition                                                                                                      |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Install           | Install the cumulative ZIP through Update Center, activate if this CMS requires explicit first-time activation, and reload. | No fatal error; plugin appears with correct version and remains active across re-install/upgrade.                   |
| Routes            | Open /super-admin, /manage, /console, /docs and /verify/random-token.                                                       | Authorized shells render; unauthorized surfaces deny safely; verify route returns neutral not-found/disabled state. |
| Migrations        | Open migration/health view twice.                                                                                           | Schema reports stable current version; no duplicate table/column errors.                                            |
| UI                | Check desktop, tablet and mobile shell.                                                                                     | Navigation and content do not overlap; no debug/status footer; no broken icons/assets.                              |
| Package           | Inspect manifest and cumulative contents.                                                                                   | Manifest/version correct; all product files present; no development secrets or temporary files.                     |

## Completion response required from coding AI

- State exact product version and schema version.
- List routes now available and their access behavior.
- List migrations/tables added.
- Provide test counts/results and any skipped tests with reasons.
- Name the cumulative ZIP and confirm Update Center compatibility.
- State known defects. If a base route, migration or activation flow is broken, mark Prompt 1 incomplete.

**PROMPT 2**

## Multi-Tenancy, Identity & RBAC

**Target release**

Suggested product version 1.1.0. Exit outcome: Organizations can be created and managed, users can hold tenant memberships, and every portal/action is enforced through tenant-aware RBAC with cross-tenant isolation.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

**Operating instructions**

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Implement the full tenancy and human authorization foundation. This prompt establishes the security boundary that every later template, certificate, form, API and asset record will depend on.

**Required implementation**

- Create tenants/organizations with unique slug, display/legal name, status active/suspended/archived, branding placeholders and timestamps.
- Create memberships linking CMS users to tenants with role and active/inactive status. Prevent duplicate active membership rows for the same tenant/user.
- Implement TenantContext that resolves the active tenant from an authorized session/membership; do not accept arbitrary tenant\_id from query/form input as proof of access.
- Implement Authorizer/Policy service with platform-admin bypass, tenant roles and granular permission keys. Keep authorization out of templates/views.
- Default human roles: Owner, Administrator, Template Designer, Issuer, Viewer. Store permission definitions centrally so custom roles can be added later without rewriting controllers.
- Recommended permission namespaces: tenant.\*, users.\*, roles.\*, templates.\*, certificates.\*, forms.\*, schedules.\*, api.\*, webhooks.\*, audit.\*, settings.\*, docs.\*.
- Create /super-admin tenant management: list, search, create, edit display details, suspend/reactivate and inspect membership summary. Do not hard-delete a tenant from normal UI.
- Create /manage tenant selector/context and tenant administration shell. Suspended tenants cannot operate tenant workflows but platform admins can inspect them.
- Create /console operator shell using the same tenant context but only operational permissions; a viewer cannot issue, a template designer does not automatically manage users.
- Create tenant member management with add existing CMS user by supported identifier, assign/change role, deactivate/reactivate membership and ownership-transfer safeguards.
- Add audit events for tenant creation/status change, membership changes, role changes, tenant switching and denied privileged actions where useful.
- Make all tenancy repositories use tenant constraints consistently and add negative tests that intentionally substitute IDs from another tenant.

**Access rules**

- Platform administrator: access /super-admin and may enter/inspect tenant context through explicit audited action.
- Tenant Owner: full tenant administration except platform-only settings; at least one active owner must remain.

- Tenant Administrator: manage most tenant configuration and members but cannot take platform actions.
- Template Designer: template/assets access only plus read-only certificate context if granted.
- Issuer: /console issuance and permitted certificate operations only; no tenant user administration by default.
- Viewer: read-only certificate/template reporting permitted by assigned policy.
- Suspended tenant: deny new issuance and management mutations; show a clear suspended state. Platform admin retains inspection/repair access.
- Inactive membership: no tenant access even if session still contains a stale tenant selection.

### Security and data rules

- Every tenant-owned table introduced now or later must have tenant\_id indexed and checked at query time.
- For direct object routes such as /manage/.../{id}, repository lookup must use WHERE tenant\_id = currentTenant AND id = requestedId or equivalent.
- Do not distinguish another tenant's object as existing in error messages. Prefer not-found or generic denial where appropriate.
- Platform-admin bypass must be explicit in the authorization service, not achieved by skipping tenant filters accidentally.
- Changing selected tenant must invalidate tenant-specific cached/session state that could leak prior tenant data.

| Verification area | Required check                                                                      | Pass condition                                                                                               |
|-------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Tenant creation   | Create Tenant A and Tenant B in /super-admin.                                       | Both are isolated, uniquely addressable and auditable.                                                       |
| Membership        | Add the same CMS user to two tenants with different roles.                          | Tenant switch works; permissions change according to selected tenant.                                        |
| Cross-tenant      | As Tenant A admin, tamper with URL/POST IDs referencing Tenant B.                   | No Tenant B data is returned or modified; response is safe and audit/test captures denial.                   |
| Suspension        | Suspend Tenant A while logged in as its admin.                                      | Mutation actions are blocked and UI clearly reports suspension; platform admin can still manage it.          |
| Role enforcement  | Test Viewer, Issuer, Designer, Admin and Owner on /manage and /console.             | Each sees only allowed navigation/actions; direct URLs are server-enforced.                                  |
| UI                | Check tenant selector, member tables and role forms at all standard viewport sizes. | No duplicate branding, broken tables or hidden critical actions; long tenant names truncate/wrap gracefully. |

**Stop condition:** Do not proceed if cross-tenant read/write can be demonstrated, if membership deactivation is not enforced on the next request, or if roles are only cosmetic UI controls.

**PROMPT 3**

## Template Domain, Assets, Variables & Versioning

**Target release**

Suggested product version 1.2.0. Exit outcome: Tenants can create reusable certificate templates with typed variables and managed assets, publish immutable versions, archive/supersede safely and preserve historical definitions.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest `development-status.md`.

**Operating instructions**

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust `tenant_id` supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update `docs/development-status.md`. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Build the non-visual template system first so the designer in Prompt 4 sits on a clean versioned data model instead of inventing persistence rules in JavaScript.

**Template model**

- Create tenant-scoped template records with name, description, category/tags, status, current draft reference, current published version reference, created/updated actor and timestamps.
- Create immutable template version records containing normalized structured layout JSON, page dimensions/orientation, variable schema, asset references, renderer options and publication metadata.
- Template states: draft/editable, published/issuable, archived/not selectable for new issuance, superseded/historical. Publishing creates a new immutable version; never mutate a published version in place.
- Allow cloning an existing template or published version into a new draft without changing the source.
- Store a deterministic canonical hash of published template version content so integrity can be checked later.
- Provide validation that rejects unknown element types, invalid dimensions, unsafe URLs and malformed variable definitions before publish.

**Variable system**

- Support system variables such as `certificate_number`, `verification_url`, `issue_date`, `expiration_date`, `tenant_name` and `recipient_name` where relevant.
- Support tenant-defined variables with key, label, type, required flag, default/sample value, visibility and validation constraints.
- Recommended types: `short_text`, `long_text`, `number`, `date`, `datetime`, `email`, `enum`, `boolean` and image reference where supported. Avoid arbitrary executable expressions.
- Variable keys are stable identifiers and should not silently change when display labels change.
- Publishing validates that every variable referenced by the layout exists and that required system elements such as QR can be inserted later without ambiguity.

**Asset management**

- Tenant-scoped assets for background, logo, seal, badge, signature and generic image.
- Server-side MIME/type and size validation; sanitize filenames; store outside directly executable web paths when possible.
- Reject unsupported SVG/vector or other active content unless a safe sanitizer/render path is explicitly implemented.
- Prevent cross-tenant asset reference. Published versions should preserve references/snapshots needed to render historically.

- Provide asset usage references so an asset used by a published version cannot be silently deleted. Use archive/replace semantics instead.

## Tenant UI

- /manage/templates: searchable/filterable template list with status and published version.
- Create/edit template metadata and variable schema screens.
- Asset library with upload, preview, usage indicator and archive controls.
- Version history page showing who published, when, version number/hash and whether certificates use it.
- Publish confirmation that summarizes validation issues and clearly states that the published version becomes immutable.

| Verification area | Required check                                                            | Pass condition                                                                                        |
|-------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Draft/publish     | Create a draft with variables and asset references, then publish v1.      | v1 is immutable and becomes selectable as published version.                                          |
| Versioning        | Edit the template after publishing and publish v2.                        | v1 record/hash remains unchanged; v2 is distinct; history shows both.                                 |
| Assets            | Try to reference Tenant B asset from Tenant A template.                   | Server rejects; no preview or metadata leak.                                                          |
| Validation        | Publish with missing variable, invalid page size and unsupported asset.   | Publish is blocked with actionable field errors; no partial published version.                        |
| Archive           | Archive template currently having published history.                      | History remains readable; template no longer appears for new issuance selection.                      |
| UI                | Check list, asset grid, variable editor and version history responsively. | Tables/cards remain usable and permission-restricted actions are not available to unauthorized roles. |

**PROMPT 4**

## Visual Certificate Designer & Preview

**Target release**

Suggested product version 1.3.0. Exit outcome: Tenant designers can visually build certificate layouts that persist as validated structured template versions and preview accurately without losing data.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

**Operating instructions**

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Build the visual editor on top of Prompt 3 storage. The editor must be reliable enough that saving, reloading, resizing and publishing do not corrupt coordinates, variable bindings or assets.

**Designer capabilities**

- Fixed-proportion certificate canvas supporting A4 landscape/portrait, Letter and custom dimensions; UI scales canvas for viewport without changing stored physical coordinates.
- Element types: static text, variable text, image/logo/seal, signature image, line, rectangle/simple shape and QR placeholder. Each element has stable ID and z-index.
- Drag, resize, align, snap-to-grid/guides, duplicate, delete, bring forward/send backward and multi-select where practical.
- Text controls: font family from approved/bundled list, font size, weight/style, color, alignment, line-height, letter spacing where renderer supports it.
- Variable picker showing required/optional fields and sample data. Designer preview resolves sample values without converting the saved element into static text.
- Background image/color and safe-area/print guides.
- Undo/redo for the active editing session and dirty-state warning on navigation/close where practical.
- Autosave may save a draft, but must not publish automatically. Manual Save Draft and Publish remain explicit.
- Preview mode that hides editor controls and renders the same normalized template definition that the local renderer will consume.
- QR placeholder always represents the system verification link; designers cannot bind it to arbitrary sensitive data.

**Persistence and safety**

- Use a schema-versioned structured JSON definition. Validate server-side on every save and publish; never trust client JSON simply because the designer created it.
- Unknown future element types must not be silently deleted when an older UI loads a template; fail safely or preserve opaque data.
- Coordinates and dimensions must round-trip with deterministic units. Avoid percentage drift after repeated saves.
- Sanitize text and metadata; do not allow arbitrary script, iframe or unrestricted HTML/CSS execution inside template elements.
- Editor load/save endpoints are tenant-scoped and permission checked. A user with templates.view may preview but not mutate.

## UI quality requirements

- Desktop-first three-zone layout: element palette, central canvas, properties inspector. On smaller screens, collapse side panels rather than squeezing the certificate into unusable width.
- Keep top toolbar compact and show save state, draft/published context and preview action without a persistent bottom status bar.
- Use sensible keyboard shortcuts only when they do not conflict with browser behavior; Delete/Backspace must not remove elements while typing in an input.
- Show validation problems inline and in a publish summary with click-to-select affected element where practical.
- Prevent image stretching surprises: preserve aspect ratio by default with explicit crop/fit controls if implemented.

| Verification area     | Required check                                                                      | Pass condition                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Round-trip            | Place multiple elements, save, reload and compare positions/sizes.                  | No drift, missing element or altered variable binding.                                                           |
| Version safety        | Open published v1 then edit by creating a draft.                                    | Published v1 remains read-only; changes go to draft/new version only.                                            |
| Unknown/invalid data  | Inject malformed/unknown element payload through request.                           | Server rejects or safely preserves according to schema policy; no fatal/data loss.                               |
| Permissions           | Viewer opens direct edit endpoint; Designer edits Tenant A and guesses Tenant B ID. | Viewer cannot mutate; cross-tenant access denied without metadata leakage.                                       |
| Canvas responsiveness | Test 1440, 1920, 768 widths and a small mobile viewport.                            | Editor remains usable; canvas scales/scrolls intentionally; properties and save/publish controls stay reachable. |
| Preview               | Compare editor preview after reload using sample data.                              | Same structure/content; QR placeholder and images are positioned correctly; no editor chrome in preview.         |

**PROMPT 5**

## Local Rendering, QR & Artifact Storage

**Target release**

Suggested product version 1.4.0. Exit outcome: Published templates render locally into deterministic PDFs with locally generated QR codes, safe tenant-scoped storage, integrity hashes and a replaceable renderer/storage abstraction.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

#### Operating instructions

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Implement all expensive/output generation locally. The certificate product must be able to render and store a certificate without calling Cloudflare, AWS or any third-party service. External renderers/storage are future adapters only.

#### Renderer architecture

- Define CertificateRendererInterface and a LocalCertificateRenderer implementation. Issuance/domain code must depend on the interface, not on a specific library.
- Inventory what PDF capability is already safely available on the host/CMS. If a library must be included, bundle a compatible dependency in the plugin so deployment does not require running Composer/CLI on the server.
- Renderer input: exact published template version snapshot, validated resolved variables, system values, asset references and output page settings.
- Renderer output: PDF bytes/temp path, page count if available, byte size, SHA-256 hash, render duration and sanitized warnings/errors.
- Make output deterministic enough that the same snapshot and data produce visually equivalent results. Do not load arbitrary remote images/fonts during normal rendering.
- Support approved local fonts and graceful fallback; missing font/asset must be surfaced as an actionable render error rather than silently producing corrupt output.

#### QR generation

- Implement a local QR generator or bundled library. No remote QR API.
- QR payload is a softcoded absolute verification URL using a high-entropy opaque token placeholder supplied by issuance.
- Set error-correction and quiet-zone settings suitable for print. Ensure minimum rendered physical size and do not permit designers to shrink below a safe threshold without warning/blocking.
- Provide a test that generates a QR and verifies the encoded value using a local decoder if available in the test environment; otherwise test generator payload and visually validate scanability in browser/manual QA.

#### Artifact storage

- Define StorageAdapter with LocalStorageAdapter as v1 default.
- Store artifacts outside publicly executable paths where possible. Access/download goes through authorization-aware controllers, not guessed filesystem URLs.
- Use tenant-separated directories or opaque object references and sanitized generated filenames; never use raw recipient input as a filesystem path.
- Store object reference, size, MIME, SHA-256, created time and render metadata in the database.

- Do not overwrite an issued PDF in place. Corrections/replacements generate new artifacts and preserve previous evidence according to lifecycle policy.
- Add health checks for base directory existence/writability and configurable free-space threshold.

### Render preview/tooling

- Add authorized template test-render action that uses sample data and creates a temporary/preview artifact with short retention or direct stream.
- A test render does not create a certificate registry record, certificate number or permanent verification token.
- Render errors show a stable error code and human-readable message; detailed internal traces go only to protected logs.

| Verification area | Required check                                                                    | Pass condition                                                                               |
|-------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| PDF output        | Test-render A4 landscape and portrait templates with text, images, shapes and QR. | Valid PDF opens, page dimensions are correct and no third-party network request is required. |
| QR                | Scan/inspect QR from rendered PDF at normal print size.                           | Resolves to the expected softcoded verification URL/token pattern.                           |
| Assets/fonts      | Render missing/corrupt asset and unavailable font.                                | Safe controlled failure/warning; no broken issued artifact presented as success.             |
| Storage isolation | Guess or substitute another tenant artifact reference.                            | Download/access denied and raw storage path is never disclosed.                              |
| Integrity         | Compare stored SHA-256 with downloaded bytes.                                     | Hash matches and metadata is recorded.                                                       |
| Failure state     | Make storage temporarily unwritable and run test render.                          | Operation fails safely; health detects problem; no false successful artifact record.         |

**PROMPT 6**

## Issuance Engine, Registry & Verification

**Target release**

Suggested product version 1.5.0. Exit outcome: A certificate can be issued end-to-end from a published template into an immutable registry record/PDF/QR and verified through configurable public or protected verification rules.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

**Operating instructions**

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Create the authoritative certificate domain and the single issuance service that every later trigger will call. This is the core product milestone: one permitted issuer can create a real certificate, download it and verify it.

**Certificate identity and registry**

- Create tenant-scoped certificate records with internal ID, human-friendly certificate number, high-entropy verification token, title/type, recipient public/display values, template ID/version, lifecycle state, source type/source reference, issuer actor, issue/expiration dates, artifact reference/hash and timestamps.
- Certificate number format must be configurable per tenant/template with safe tokens such as year and sequence. Allocate sequence transactionally to prevent duplicate numbers under concurrency.
- Verification token must be random/non-sequential with sufficient entropy and unique database index. Never derive it predictably from certificate number.
- Store an immutable issuance snapshot containing all resolved values, exact template version/hash, source metadata needed for audit, and renderer metadata. Sensitive private source data should not be duplicated unnecessarily.
- Do not mark ISSUED until required database record, render and final storage succeed. Failure state preserves enough information for authorized retry without generating uncontrolled duplicates.

**CertificateIssuanceService**

1. Resolve authenticated actor and TenantContext.
2. Authorize certificates.issue for the requested published template/source.
3. Load published immutable template version with tenant constraint.
4. Normalize and validate required variable values and server-generated system values.
5. Apply deduplication/idempotency hook if the source supplies one.
6. Reserve certificate number and verification token transactionally.
7. Create issuance snapshot and pre-issued/pending state.
8. Render QR and PDF using Prompt 5 interfaces.
9. Persist artifact metadata and integrity hash.
10. Commit final ISSUED state and write audit event.
11. Return normalized certificate result for UI/API callers.

**Verification management**

- Tenant verification modes: public, access-code protected, authenticated-user only, and disabled. Mode is tenant default with optional template override where policy permits.

- Route `/verify/{token}` resolves only by opaque token and authoritative registry state. It must not trust claims encoded in the PDF/QR.
- Public result can show status, certificate number/title, issuing organization, recipient display/masked value, issue/expiration date and replacement reference. Private email, employee ID, payload JSON, internal IDs and audit metadata remain hidden.
- Access-code mode stores a secure hash and uses constant-time/secure verification. Rate-limit failed code attempts using lightweight database/session controls; do not need Redis at expected scale.
- Authenticated mode requires CMS login plus a policy mapping (for example recipient-owned or tenant member) defined clearly in implementation.
- Disabled verification returns a neutral tenant-configured unavailable message rather than leaking whether a token is valid.
- Verification page should support noindex policy and tenant branding without allowing unsafe HTML injection.

## Management UI

- `/console` issue screen for authorized issuer: select published template, enter variables, preview summary, issue, then show certificate number/download/verification link.
- `/manage` certificate registry: list, search and filter by number, recipient, template, status and date. Sensitive values shown according to permission.
- Certificate detail page shows authoritative metadata, template version, artifact hash/status, source type, verification mode and audit timeline placeholder.
- Do not expose raw verification token in list tables unnecessarily; copy verification link from detail through authorized action.

| Verification area   | Required check                                                    | Pass condition                                                                                           |
|---------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| End-to-end issue    | Issue one certificate from <code>/console</code> .                | One registry record, one number, one token and one final PDF; status ISSUED only after storage succeeds. |
| Historical template | Issue from v1, publish v2, re-open old certificate.               | Old record/PDF remains tied to v1 snapshot and does not change.                                          |
| Verification        | Scan/open QR token in public mode.                                | Correct certificate status and allowed public fields; no internal IDs/paths.                             |
| Protected modes     | Test access-code, authenticated and disabled modes.               | Each follows policy and cannot be bypassed by direct URL/request manipulation.                           |
| Concurrency         | Simulate two issuance requests targeting same numbering sequence. | Unique certificate numbers; no duplicate token; transaction remains consistent.                          |
| Render failure      | Force renderer/storage failure mid-issuance.                      | No falsely ISSUED record; retry behavior is controlled and auditable.                                    |

**PROMPT 7**

# Certificate Lifecycle, Audit, Search & Operational Management

**Target release**

Suggested product version 1.6.0. Exit outcome: Issued certificates can be safely revoked, expired, cancelled, replaced/reissued and investigated with complete tenant-scoped history, search and audit controls.

## COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

### Operating instructions

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Turn issuance into a maintainable registry with explicit lifecycle semantics. Never “edit history” silently; corrective actions create state transitions and linked records.

### Lifecycle state machine

- Define allowed states/transitions such as PENDING/FAILED before issue and ISSUED, REVOKED, EXPIRED, REPLACED, CANCELLED after appropriate operations. Use the exact minimal set that matches implementation, document it and reject invalid transitions.
- Revocation requires reason, actor and timestamp. Verification immediately reflects REVOKED and may show a tenant-approved public reason category without exposing internal notes.
- Expiration is driven by expiration\_date and/or explicit state normalization. Verification must report expired once the date passes even if a scheduled state update has not run.
- Replacement/reissue creates a new certificate/number/artifact and links old -> replacement and new -> replaces. Old verification reports REPLACED and may link to the new verification record if policy allows.
- Cancellation applies only where business meaning is clear (for example a pre-issued/pending record). Do not use hard delete as a substitute for lifecycle.
- Correction rules must distinguish metadata/admin notes that may be editable from immutable issuance facts. If recipient/title data needs correction after issue, prefer replacement rather than silent mutation.

### Audit system

- Audit events should include tenant, actor type/id, action, target type/id, timestamp, safe metadata, source IP/user agent where appropriate and correlation/request ID.
- Audit at minimum: template publish/archive, certificate issue/failure/retry, revoke, replace, verification policy change, artifact download if security-sensitive, membership/role changes, API client changes later, schedule/webhook changes later.
- Audit records are append-only through normal application flows. Provide filters/export subject to permission. Never include full API secrets, access codes or raw sensitive payloads.
- Platform admin viewing cross-tenant audit data must be explicit and auditable.

### Operational search and detail

- Fast indexed search by certificate number, recipient display/search field, template, status, issue date range and source.
- Pagination must be server-side and stable; do not load all tenant certificates into browser memory.
- Certificate detail shows lifecycle timeline, replacement linkage, render/artifact integrity metadata, verification policy and audit events according to access rights.

- Authorized export can produce CSV of registry metadata. Export must apply same filters/permissions and should avoid exposing private payload fields by default.

| Verification area   | Required check                                                                                     | Pass condition                                                                             |
|---------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Revoke              | Revoke an issued certificate with reason.                                                          | Registry/audit updated once; verify page changes immediately; PDF itself is not rewritten. |
| Replace             | Replace certificate due to corrected recipient/title value.                                        | New certificate is issued; old one becomes REPLACED; both remain traceable.                |
| Expire              | Set expiration in past/future and test verification.                                               | Status presentation follows date correctly even without external scheduler.                |
| Invalid transitions | Attempt revoke twice, replace revoked record if policy disallows, or edit immutable issue payload. | Server rejects with stable error and no state corruption.                                  |
| Search              | Create enough test records for pagination/filter combinations.                                     | Correct tenant-only results, stable pagination and appropriate indexes.                    |
| Audit privacy       | Inspect audit as tenant admin and viewer.                                                          | Only authorized roles can view; secret/sensitive payloads are absent.                      |

**PROMPT 8**

# Manual Issuance, Dynamic Forms, Automation & Scheduling

**Target release**

Suggested product version 1.7.0. Exit outcome: Tenants can trigger issuance manually, through configurable forms, and through local database-backed schedules/rules without separate business logic or external queues.

## COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

### Operating instructions

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Add multiple human and automatic trigger sources while keeping CertificateIssuanceService as the only authoritative issuance path.

### Manual issuance refinement

- Improve /console manual issuance with template search, dynamic fields generated from template variable schema, validation, preview summary and clear success/failure state.
- Optional draft entry may be saved without allocating final certificate number/token. Final issuance is always explicit unless the configured source is automatic.
- Issuer sees only templates permitted/active for the current tenant and cannot override protected system variables such as verification token or certificate number.

### Dynamic form builder

- Tenant admins can create forms mapped to a published template and map each form field to a template variable or metadata field.
- Field types align with variable types; support labels, help text, required flag, options, validation and hidden server-controlled values. No arbitrary server code/expressions.
- Form visibility modes: tenant-authenticated, private token/link, and public if explicitly enabled. Public forms require lightweight abuse controls such as CSRF where session-based, honeypot/rate limiting and validation.
- Form submission policy: issue immediately, require approval, or create request only. Approval requires a user with certificate issuance permission and records actor/time.
- Submissions have their own tenant-scoped records/status and do not become issued certificates unless CertificateIssuanceService succeeds.
- Allow prefill through signed/opaque token only if implemented safely; never let query parameters override protected server values silently.

### Automation and scheduling

- Create schedule/rule records with tenant, trigger type, template version policy, field mapping/input source, run\_at/recurrence, timezone, enabled status, next\_run, last\_run/result and idempotency identity.
- Use a database-backed job table with status, run\_after, lease token/owner, lease\_until, attempts, last error code and timestamps. No Redis/queue service.

- Provide web-safe execution options: protected HTTP scheduler endpoint suitable for a hosting control-panel cron if available, opportunistic/lazy runner on normal application traffic with strict time budget, and authorized “Run due jobs” button. No CLI requirement.
- Atomic lease acquisition prevents overlapping web requests from executing the same due job twice.
- Automatic trigger types in v1 should remain bounded: scheduled one-time/recurring issuance from stored approved payload, form approval action, or internal rule event explicitly supported by the product. Do not implement arbitrary code execution.
- Schedules must call CertificateIssuanceService with an idempotency key derived from schedule occurrence so retries do not create duplicates.
- Tenant suspension or disabled schedule prevents execution. Failed jobs retry according to a small configurable policy and surface clearly in /manage.

## UI requirements

- /manage/forms list/builder/submissions/approval queue.
- /manage/schedules list, create/edit, next run, last result, pause/resume, run-now and failure details.
- /console manual issuance remains fast and simple; form/schedule administration belongs in /manage.
- Use clear source labels on certificate detail: manual, form, schedule/automation, API, bulk.

| Verification area | Required check                                                         | Pass condition                                                                                                  |
|-------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Manual            | Issue through /console with required/optional variables.               | Dynamic validation and issuance use central service; protected system values cannot be forged.                  |
| Form immediate    | Submit an enabled form configured for immediate issuance.              | Submission and one certificate created; errors preserve submission state without duplicate issue.               |
| Form approval     | Submit then approve as authorized issuer; try as viewer.               | Authorized approval creates one certificate; viewer cannot approve/direct-call endpoint.                        |
| Scheduled run     | Create due schedule and invoke two runners nearly simultaneously.      | Atomic lease allows one occurrence/one certificate only.                                                        |
| Retry/idempotency | Force render failure then retry job occurrence.                        | At most one final certificate for that occurrence; failure and retry are auditable.                             |
| UI                | Test form builder, approval queue and schedule pages across viewports. | Long field labels/options and schedules remain usable; no accidental nested horizontal scrolling on page shell. |

**PROMPT 9**

## API, Authentication, Idempotency, Webhooks & Integrations

**Target release**

Suggested product version 1.8.0. Exit outcome: External applications can securely issue/query/manage permitted certificates through versioned tenant-scoped APIs, and receive signed webhooks, without human CMS sessions.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

**Operating instructions**

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Create an integration surface suitable for internal and external applications. API clients are separate identities from human users and receive only explicit scopes.

**API client and authentication model**

- Create tenant-scoped API clients with name, client/key ID, secret hash, status, scopes, created/last-used/rotated/revoked timestamps and optional environment label.
- Reveal a newly generated secret exactly once. Never store or display the full secret later. Provide rotate/revoke flow that invalidates previous secret according to documented grace policy if any.
- Use Authorization: Bearer <credential> or a similarly standard header. Parse key ID separately from secret if useful for lookup. Compare secret securely; do not put credentials in query strings.
- API client authentication resolves exactly one tenant. A request body tenant\_id never changes the credential's tenant.
- Recommended scopes: certificates.issue, certificates.read, certificates.revoke, certificates.replace, templates.read, verification.read, forms.submit, webhooks.manage as appropriate. Follow least privilege.
- Rate limiting can be lightweight/database/in-memory per request process at this low scale, but architecture should allow a future adapter. Primary protections are strong secrets, scopes and tenant isolation.

**Versioned API contract**

```
POST /api/v1/certificates
GET /api/v1/certificates/{id-or-number}
GET /api/v1/certificates/{id-or-number}/download
POST /api/v1/certificates/{id-or-number}/revoke
POST /api/v1/certificates/{id-or-number}/replace
GET /api/v1/templates
GET /api/v1/templates/{id}/schema
GET /api/v1/verification/{token} (only if tenant policy permits API verification)
POST /api/v1/forms/{form-key}/submit (if enabled for API use)
```

- Use consistent JSON envelope/error format with stable machine error code, human message, correlation/request ID and field validation details where safe.
- Use appropriate HTTP status codes; do not leak whether another tenant's ID exists.
- Certificate create API calls CertificateIssuanceService; it cannot bypass template publication, numbering, rendering, lifecycle or audit rules.

- Document pagination, filtering and date formats. Use ISO 8601 timestamps and explicit timezone behavior.
- Provide template schema endpoint so integrators can discover required custom fields without scraping the UI.

### Idempotency and replay safety

- Support Idempotency-Key for certificate creation/replacement and other appropriate mutation endpoints.
- Store tenant + API client + endpoint/operation + idempotency key + request fingerprint + result reference for a defined retention period.
- Same key + same logical request returns the original result without creating a duplicate certificate.
- Same key + materially different request returns an explicit idempotency conflict rather than silently reusing or issuing again.
- Concurrent duplicate requests must be serialized/uniquely constrained so two certificates cannot be created before the first result is stored.

### Webhooks

- Tenant admins can configure webhook endpoints, subscribed events, status and secret. Recommended events: certificate.issued, certificate.failed, certificate.revoked, certificate.replaced, certificate.expired if emitted, form.submitted/approved if useful.
- Sign webhook payloads with HMAC over timestamp + raw body (or another clearly documented canonical scheme). Include delivery ID, event ID, timestamp and event type.
- Reject dangerous endpoint schemes; only HTTPS by default in production. Add basic SSRF protections appropriate to the runtime, including blocking obvious loopback/link-local/private destinations unless an explicit trusted internal mode is designed.
- Deliver synchronously only if it will not hold issuance hostage; preferable local database-backed delivery records with a short web-triggered retry runner similar to Prompt 8. Certificate issuance should not be rolled back merely because a webhook receiver is down.
- Record attempts, response status/duration and sanitized error. Never mark a webhook delivery successful without an actual successful HTTP attempt/response according to policy.
- Provide retry/backoff and manual retry. Do not expose webhook secret after creation except rotate/reveal-new flow.

### API documentation

- /docs must include authentication, client creation, scopes, curl-style examples, request/response schemas, idempotency behavior, error codes, webhook verification and migration/versioning policy.
- Examples must use placeholders, not real production secrets/domains. Generated examples should use softcoded base URL from current environment where practical.
- Document that API certificates are tenant scoped and verification/privacy policy may affect fields returned.

| Verification area | Required check                                                                    | Pass condition                                                                             |
|-------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Auth              | Call API with valid, invalid, revoked and wrong-tenant credentials.               | Valid client sees only its tenant; invalid/revoked denied; no tenant existence leak.       |
| Scopes            | Use read-only client to issue/revoke and issue-only client to manage API clients. | Every unauthorized operation denied server-side.                                           |
| Idempotency       | Send same create request/key twice and concurrently.                              | One certificate; replay returns same logical result; changed body with same key conflicts. |
| Webhook           | Receive a signed issued event, then simulate timeout/500 and retry.               | Signature verifies; failures remain failed/retryable; no false "done" without HTTP call.   |
| SSRF              | Attempt loopback/private/file/non-HTTP webhook destinations.                      | Rejected according to policy; safe validation error.                                       |
| Docs              | Follow documentation using a newly created API client.                            | Examples work against current API and match actual response/error behavior.                |

**PROMPT 10**

## Bulk Issuance & Imports

**Target release**

Suggested product version 1.9.0. Exit outcome: Tenants can import and issue large recipient sets through paste/CSV/XLSX using safe server-side chunks, resumable progress and row-level outcomes without an arbitrary 500-user-style ceiling.

### COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

**Operating instructions**

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Add bulk issuance without introducing a separate queue infrastructure. The browser orchestrates safe chunks and the server persists batch/row state, while each row still uses CertificateIssuanceService.

**Import sources and mapping**

- Support CSV, XLSX if a safe bundled/existing parser is available, and paste/tabular input. Do not require external conversion service.
- No arbitrary business ceiling such as 500 rows. Respect actual server upload/request limits and process data incrementally so a large import does not require one huge PHP request.
- Step 1 upload/ingest -> Step 2 preview headers/sample -> Step 3 map columns to template variables -> Step 4 validate -> Step 5 start batch -> Step 6 chunked issuance -> Step 7 results/export.
- Allow reusable mapping profiles per tenant/template where useful.
- Required variables, types and enum options come from the published template schema; do not invent separate validation rules.

**Batch persistence and execution**

- Create batch record with tenant, source file/object, template/version, actor, mapping, counts, status, created/started/completed times.
- Create row records or durable normalized row state with row number, idempotency identity, validation status, certificate reference if issued, error code/message and retry state.
- Process server-side chunks sized conservatively by configuration/time budget. Browser polls/requests next chunk and shows live completed/total/issued/failed/skipped counts.
- Reloading the browser must recover progress from the server rather than restarting from row 1.
- Retry failed rows only, without reissuing successful rows. Use per-row idempotency key derived from batch + row identity.
- Allow cancel/pause between chunks. Do not interrupt a row transaction halfway and label it cancelled if issuance actually succeeded.
- Keep imported source/private data under tenant access controls and retention policy.

**UX and error handling**

- Preview the first rows and mapping before issue. Show exact published template version that will be used.
- Validation stage should catch obvious missing/invalid values before spending render time, while final server validation still occurs during issuance.

- Results table shows row number, recipient key/display value, status, certificate number if issued, and actionable error message/code.
- Download failed/skipped rows as CSV with original row reference plus safe error columns.
- Live progress is AJAX/fetch-based and accessible; do not rely only on animated visuals. Provide numeric counts and final summary.

| Verification area | Required check                                                             | Pass condition                                                                                                    |
|-------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Large import      | Import a file well above 500 rows in the test environment.                 | No hard 500-row rejection; work proceeds in chunks within server limits.                                          |
| Resume            | Close/reload browser mid-batch.                                            | Progress is recovered; already successful rows are not issued again.                                              |
| Validation        | Include missing required values, bad dates and unknown enum options.       | Rows fail individually with clear errors; valid rows can continue according to selected mode.                     |
| Cross-tenant      | Try to reuse batch/file/row ID from another tenant.                        | No data or progress leakage.                                                                                      |
| Retry             | Repair input/allowed values if supported and retry failed rows.            | Successful rows remain untouched; failed rows can complete once; counts reconcile exactly.                        |
| UI                | Test upload, mapping, progress and result tables on desktop/tablet/mobile. | Mapping remains understandable; progress is readable; tables use responsive treatment without page-wide breakage. |

**PROMPT 11**

# Dashboards, Branding, Documentation & Admin Operations

**Target release**

Suggested product version 1.10.0. Exit outcome: The product is feature-complete for daily tenant/platform operation with polished dashboards, branding/settings, reporting/export, health tools and accurate /docs.

## COPY/PASTE AI DEVELOPMENT PROMPT

Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.

### Operating instructions

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Consolidate the platform into an enterprise daily-use product. This prompt should not introduce a second implementation of core flows; it should expose, document and operationalize what Prompts 1-10 already built.

### Tenant branding and settings

- Tenant display/legal name, logo/mark, verification page branding, optional issuer identity/contact, default verification mode, certificate numbering settings and privacy defaults.
- Keep global product identity and tenant/workspace identity visually distinct so branding is not duplicated in the header.
- Validate uploaded branding assets and apply sensible size constraints/cropping. Do not let tenant custom HTML/JS enter platform pages.
- Settings changes are permission checked and audited. Some changes, such as verification visibility, should show impact warning.

### Dashboards and reports

- /manage dashboard: issued today/month, active/revoked/expired counts, recent failures, most-used templates, upcoming schedules, webhook/API health summary and storage/renderer health for authorized admins.
- /console dashboard: quick issue action, recently issued by current user/team where permitted, pending approvals and failures needing operator action.
- /super-admin dashboard: tenant counts/status, product version/schema, migration/storage/renderer health, recent platform-level failures and aggregate issuance volume without exposing tenant private payloads.
- Server-side search/filter/export for certificates, templates, forms, batches, schedules, API clients/webhooks and audit where appropriate.
- Exports respect permission and current tenant/filter; do not include secrets or unnecessary private fields.

### /docs completion

- Developer docs: architecture overview, authentication, API client creation, scopes, endpoint reference, examples, error codes, idempotency, webhook signing, versioning and integration patterns.
- Administrator docs: tenant/member roles, template creation/versioning, visual designer, assets, manual issue, lifecycle operations, verification privacy, forms, schedules, API clients, bulk imports, audit and troubleshooting.
- Operator docs: /console issuance, approvals, retry/failure handling and certificate lookup.
- Verification docs: QR behavior, public/access-code/authenticated/disabled modes and what fields are exposed.
- Deployment docs: Update Center install/upgrade, first-time activation behavior, migration status, local storage permissions, backup/restore expectations and no-CLI operation.

- Docs must describe actual implemented behavior. Remove stale planned endpoints/features that do not exist.

## Operational tooling

- Health page checks DB/schema, storage writability/free-space threshold, renderer availability, QR capability and scheduler/webhook backlog indicators.
- Safe test actions: render sample certificate, send webhook test event if configured, run due jobs, validate storage. These are permission checked and audited.
- Provide clear status/version information in dedicated admin pages, not persistent footer strips.
- Add data retention/cleanup controls for temporary previews, failed import source files and old webhook attempt logs while preserving issued certificate/audit evidence according to policy.

## UI polish

- Consistent navigation hierarchy across /super-admin, /manage and /console; active tenant/workspace identity visible without duplicated product logo/name.
- Use compact enterprise tables, filters and pagination; maintain reasonable density on ultra-wide screens.
- Empty states guide the next action but do not hide navigation or create giant decorative whitespace.
- Modals/drawers/forms have visible validation, accessible labels, keyboard operation and destructive-action confirmation.
- Search inputs used for large lists should debounce or submit intentionally; do not issue a database request on every keystroke without control.

| Verification area | Required check                                                                                          | Pass condition                                                                                      |
|-------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Daily-use flow    | From tenant dashboard create/publish template, issue certificate, find it, verify it and inspect audit. | Navigation is coherent and no hidden admin-only dependency blocks normal tenant operation.          |
| Branding          | Configure two tenants with different logos/names.                                                       | Each tenant sees its own branding/verification presentation; product branding is not duplicated.    |
| Health            | Break storage permission in test environment or simulate renderer failure.                              | Health shows actionable failure to authorized admins; ordinary users do not see internals.          |
| Docs              | Follow install + API + manual issuance docs from a clean user perspective.                              | Instructions match actual routes, scopes and current UI.                                            |
| Exports           | Export filtered certificates/audit data as authorized and unauthorized roles.                           | Correct filtered tenant data only; unauthorized roles denied; secrets/private payloads absent.      |
| Responsive UI     | Run full universal UI matrix across all major portals.                                                  | No major visual regression, duplicate branding, unusable table, clipped navigation or debug footer. |

**PROMPT 12**

# Production Hardening, Security, Performance, Recovery & Final Release

**Target release**

Suggested product version 2.0.0 / production. Exit outcome: The entire planned v1 passes fresh-install/upgrade, multi-tenant security, lifecycle, API, browser, performance and recovery gates and ships as the final cumulative production package.

## COPY/PASTE AI DEVELOPMENT PROMPT

*Copy this prompt section into the coding AI together with access to the current repository and the latest development-status.md.*

### Operating instructions

- Continue from the existing repository. Establish ground truth from code, schema/migrations, manifest/package metadata and the full test suite before making changes. Do not rely only on prior narrative reports.
- This product must remain a modular multi-tenant SOI CMS application, deployed through the Update Center, with no required SSH/CLI, Redis, external queue, Cloudflare/AWS service or hardcoded domain.
- Preserve all working functionality from prior prompts. Do not replace stable modules unnecessarily. Fix regressions discovered while implementing this scope.
- All tenant-owned data must be tenant-scoped at the repository/service query boundary. Never trust tenant\_id supplied by a browser/API request as authorization.
- Use CMS authentication/session/CSRF for browser surfaces and the product authorization layer for permissions. Return safe errors with no stack traces or SQL details.
- At completion, create exactly one cumulative Update Center-compatible ZIP for this release and update docs/development-status.md. The ZIP must contain all required current product files, not just the files changed in this prompt.

Objective: Treat the repository as a release candidate and try to break it. Do not add large new product features. Fix defects, close security gaps, improve performance/indexes, validate recovery, update documentation and produce the final cumulative ZIP.

### Ground-truth audit before edits

- Inventory all routes, controllers, services, modules, database tables/indexes, migrations, permissions, API endpoints, scheduled jobs, webhooks, docs and tests.
- Compare implementation against Prompts 1-11 and docs/development-status.md. Classify each item complete, partial, missing, obsolete or defective.
- Run the full suite before making changes and record the baseline. Do not delete failing tests merely to obtain green results.
- Search for hardcoded domains, direct tenant\_id trust, raw SQL interpolation, unsafe file paths/uploads, secrets in logs, debug routes, TODO stubs and dead navigation.

### Security hardening

- Systematically test IDOR/cross-tenant access for tenants, memberships, templates, versions, assets, certificates, artifacts, forms/submissions, schedules/jobs, batches/rows, API clients, webhooks and audit events.
- Verify every state-changing browser route has CSRF protection and correct permission. Verify API does not depend on CMS CSRF and instead uses client auth/scopes.
- Review output escaping, template sanitization, upload MIME validation, path traversal defenses, content security policy/headers, verification privacy and noindex policy.
- Review API secret storage/rotation, access-code hashes, HMAC webhook signing/replay timestamp policy, webhook SSRF restrictions and idempotency concurrency.
- Ensure platform-admin bypass is explicit/audited and does not mask missing tenant predicates in ordinary repositories.
- Ensure user-visible errors never contain stack traces, SQL statements, local paths or credential material.

### Performance and reliability

- Review indexes using actual query patterns: tenant\_id + status/date, unique verification token, tenant + certificate number, tenant + template/status, memberships, API key lookup, idempotency, due jobs, webhook retry, batch status.
- Eliminate obvious N+1 list/dashboard queries and unbounded SELECTs. All large registries use server-side pagination.

- Measure local render time and memory for representative templates and verify low daily volume is comfortably handled without external infrastructure.
- Validate transactional number allocation, concurrent issuance, schedule leases, idempotency races and import chunk recovery.
- Validate storage health/free-space warnings and behavior when disk is unwritable/full-like in test conditions.
- Verify a third-party outage cannot affect core v1 issuance because no third-party runtime dependency is required.

### Fresh install, upgrade and recovery

- Fresh install the final cumulative ZIP on a clean supported CMS database, activate if required, run migrations through web path and execute core smoke tests.
- Upgrade from the previous cumulative release and at least one earlier representative release if practical. Preserve activation, tenant data, template versions, certificates, artifacts, API clients/scopes, forms, schedules and audit.
- Verify migration rerun/idempotency and recovery from a deliberately failed migration in test environment without manual SQL as the normal path.
- Document backup set: database + local certificate/artifact storage + tenant assets/config needed for restore. Test a representative restore into a test environment if tooling allows.
- Run integrity scan that detects missing artifact referenced by DB and orphan/temporary artifacts where safe; do not auto-delete issued evidence.

### Final browser/E2E scenarios

- Platform admin creates Tenant A and Tenant B; membership roles are assigned.
- Tenant A designer creates assets/template variables, designs template and publishes v1.
- Issuer manually issues certificate; PDF downloads and QR verifies according to public policy.
- Designer publishes v2; old certificate remains tied to v1.
- Revoke one certificate and replace another; verification states update correctly.
- Create private/access-code and authenticated verification examples; attempt bypass.
- Build form with approval; submit and approve; one certificate is issued.
- Create a due schedule; concurrent runner attempts execute one occurrence only.
- Create scoped API client; issue with Idempotency-Key; replay is safe; cross-tenant guesses fail.
- Configure signed webhook; success and failure/retry behavior are accurate.
- Run a bulk import above 500 rows; reload during processing; final counts reconcile.
- Suspend Tenant A and verify new mutations/issuance stop while platform inspection remains available.
- Run UI verification on desktop, ultra-wide, tablet and mobile management surfaces.
- Fresh-install and upgrade final ZIP; repeat critical smoke routes.

### Final acceptance criteria

- [ ] All 12 planned prompts are implemented and represented accurately in code/docs.
- [ ] No known critical/high-severity defects remain.
- [ ] /super-admin, /manage, /console, /docs and verification routes work for intended roles.
- [ ] Template versioning and immutable issuance snapshots prevent historical drift.
- [ ] Local rendering, local QR and local storage work without third-party services.
- [ ] Manual, form, automatic/scheduled, API and bulk triggers all use CertificateIssuanceService.
- [ ] Verification modes and privacy are enforced server-side.
- [ ] Lifecycle revoke/expire/replace behavior is authoritative and immediate.
- [ ] Cross-tenant negative tests pass for every tenant-owned resource.
- [ ] API auth/scopes/idempotency and webhook signing/retry/SSRF rules pass tests.
- [ ] Fresh install and supported upgrade work through Update Center without manual SQL/SSH/CLI.
- [ ] Full automated suite and browser/E2E suite are green or any environment-only skips are explicitly justified.
- [ ] Final cumulative production ZIP and final development-status/release notes are produced.

| Verification area | Required check                                                                   | Pass condition                                                                  |
|-------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Fresh install     | Install final ZIP on clean CMS and run complete smoke/E2E.                       | No missing migration/file/route and no manual server intervention.              |
| Upgrade           | Upgrade prior working installation containing real test data.                    | Data, activation, artifacts and permissions preserved; migrations succeed once. |
| Security          | Run tenant/role/API/CSRF/upload/verification/webhook negative suites.            | No confirmed high-severity authorization/data exposure issue.                   |
| Performance       | Exercise list/search/dashboard, batch and render paths with representative data. | No unbounded query/memory behavior; expected few-certificates-per-day load has  |

| Verification area | Required check                                                    | Pass condition                                                                                         |
|-------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
|                   |                                                                   | ample headroom.                                                                                        |
| UI regression     | Capture/inspect all major screens at standard viewports.          | No broken shells, inaccessible action, overflow regression, duplicate branding or status footer strip. |
| Recovery          | Verify backup/restore instructions and artifact integrity checks. | A developer/admin can recover the authoritative registry and files using documented steps.             |

**Final stop condition:** If any critical/high defect, cross-tenant leak, unreliable issuance state, broken upgrade/fresh install or materially incomplete core feature remains, do not label the product 100% complete. Produce a Prompt 12.1 repair package and repeat the final gate.

# Repair Prompt Template (N.1) - Not Counted as a Main Roadmap Prompt

Use this whenever testing after a main prompt finds a defect. A repair prompt fixes the current phase; it does not add new roadmap scope.

Continue from the exact repository state produced by Prompt N.

A production verification defect was found after installing the cumulative ZIP. Fix the defect completely before any next roadmap prompt begins.

Rules:

1. Establish root cause from code, database state, logs/tests and reproduction steps. Do not patch only the visible symptom.
2. Preserve all unrelated working behavior and tenant data.
3. Add a regression test that fails before the fix and passes after it.
4. Re-run the full available automated suite plus affected browser/HTTP checks.
5. Re-run cross-tenant and authorization checks for the affected resource.
6. Increment the product patch version; increment schema version only if schema changes.
7. Produce one new cumulative Update Center ZIP containing the complete current product.
8. Update docs/development-status.md with root cause, fix, tests, versions and whether Prompt N can now be considered complete.
9. If the defect cannot be safely fixed in this run, stop and report the exact blocker. Do not move to Prompt N+1.

Defect reproduction:

<PASTE EXACT STEPS / SCREENSHOT NOTES / ERROR>

Expected behavior:

<PASTE EXPECTED RESULT>

## Final Handoff Checklist for Every AI Run

- [ ] Exact product version and schema version reported.
- [ ] Files/modules/migrations changed are summarized.
- [ ] Routes/endpoints added or changed are listed.
- [ ] New permissions/scopes are listed.
- [ ] Automated test totals and browser/HTTP verification results are reported.
- [ ] Known defects, partial items and deliberately out-of-scope items are explicit.
- [ ] Cumulative ZIP filename/path is given and package validation is reported.
- [ ] development-status.md contains current state and next prompt instructions.
- [ ] The response does not claim completion if a required exit gate failed.

## Planned v1 Completion Definition

The roadmap contains 12 planned main prompts. The website/application reaches the planned v1 “complete” state when Prompt 12 passes the full final acceptance gate. The number of actual AI runs may be higher because repair prompts are intentionally inserted whenever testing finds defects. This is the safer process: quality gates determine progression, not an arbitrary promise that exactly twelve executions will be bug-free.

### Delivery principle

Do not skip verification to preserve the prompt count. The latest cumulative ZIP should always be the only package needed for a fresh supported installation, and every release should be safe to hand to another developer/AI using development-status.md as the ground-truth handoff.